

INTERNET DAS COISAS

1. A Estrutura Básica (O "Main" Oculto)

Em Python, seu código roda de cima abaixo (ou dentro de um `while True`). No Arduino, a estrutura é obrigatória e dividida em duas funções principais:

- **setup():** Equivalente a um bloco de inicialização que roda **uma única vez** quando o Arduino é ligado ou resetado (configurações de pinos, baud rate, etc.).
- **loop():** Equivalente a um `while True`: infinito. O código aqui dentro roda repetidamente sem parar.

C++

```
void setup() {  
    // Configurações iniciais (roda 1x)  
}  
  
void loop() {  
    // Código principal (roda em loop infinito)  
}
```

2. Variáveis e Tipos de Dados (Tipagem Estática)

Enquanto no Python você apenas escreve `x = 10`, no Arduino (C++) você **precisa declarar o tipo da variável**.

Tipo em C++	O que é	Exemplo em Python (equivalente conceitual)
int	Número inteiro (-32.768 a 32.767)	<code>x = 10</code>
float	Número com ponto flutuante/decimal	<code>pi = 3.14</code>
bool	Booleano (true ou false)	<code>ativo = True</code> (atenção às minúsculas!)
char	Um único caractere	<code>letra = 'A'</code>
String	Texto (com S maiúsculo)	<code>nome = "Arduino"</code>

C++

```
int pinoLed = 13;          // Inteiro  
  
float temperatura = 23.5; // Decimal  
  
bool estado = false;      // Booleano (true/false)
```

3. Controle de Fluxo (Condicionais e Loops)

A sintaxe para `if`, `else`, `for` e `while` é praticamente idêntica à do Python, com a única diferença de que **usamos chaves {}** para delimitar blocos de código em vez de indentação, e **ponto e vírgula ;** no final das instruções.

C++

```
// If / Else (Igual ao Python, mas com chaves)

if (temperatura > 30) {

    // Faça algo ;

} else {

    // Faça outra coisa ;

}
```

4. Funções Essenciais de Entrada e Saída (I/O)

Aqui está o coração do Arduino interagindo com o mundo físico.

- **pinMode(pino, modo):** Define se o pino vai funcionar como entrada (INPUT) ou saída (OUTPUT). *Pense nisso como configurar a direção do fluxo.*
- **digitalWrite(pino, valor):** Envia um sinal digital (HIGH / LOW, ou seja, 5V ou 0V) para um pino.
- **digitalRead(pino):** Lê o estado digital de um pino (HIGH ou LOW).
- **analogRead(pino):** Lê o valor de um pino analógico. Retorna um número inteiro de **0 a 1023** (mapeando a tensão de 0V a 5V) — equivalente a uma resolução de 10 bits.
- **analogWrite(pino, valor):** Simula uma saída analógica via PWM (Pulsos Modulados). O valor varia de **0 a 255** (0 = 0%, 255 = 100% do ciclo de trabalho).

C++

```
void setup() {

    pinMode(13, OUTPUT); // Configura o pino 13 como saída
    pinMode(7, INPUT);   // Configura o pino 7 como entrada

}

void loop() {

    digitalWrite(13, HIGH); // Liga o LED (5V)

    delay(1000); // Espera 1 segundo

    digitalWrite(13, LOW);  // Desliga o LED (0V)

    delay(1000);

}
```

5. Funções de Tempo e Depuração (Serial)

- **delay(ms):** Pausa a execução do programa pelo tempo especificado em milissegundos (equivalente ao `time.sleep()` do Python). **Aviso:** pausa o microcontrolador inteiro.
- **millis():** Retorna o tempo em milissegundos desde que a placa foi ligada. Muito usado para criar temporizações sem travar o código (equivalente mental a rodar threads ou relógios assíncronos, embora seja síncrono).
- **Serial.begin(taxa):** Inicializa a comunicação serial (geralmente 9600 bauds).
- **Serial.println(dado):** Imprime uma mensagem no Monitor Serial (exatamente como o `print()` do Python).

C++

```
void setup() {  
    Serial.begin(9600); // Inicia o "print" no baud rate 9600  
}  
  
void loop() {  
    int leitura = analogRead(A0);  
    Serial.print("Leitura do sensor: ");  
    Serial.println(leitura); // Imprime e pula a linha (como print())  
    delay(500);  
}
```

Dica de Ouro para Pythonistas:

No Python, você não se preocupa com o tamanho das variáveis na memória. No Arduino (especialmente em placas menores como o Uno), a memória RAM é extremamente limitada (geralmente 2KB a 8KB). Por isso, prefira usar byte (0 a 255) ou int para números pequenos, economizando preciosos bytes da placa.

Gostaria de ver um exemplo prático comparando um script simples em Python com o seu equivalente exato em Arduino para fixar a lógica?